

BLAZEDEMO AUTOMATION PROJECTCODE

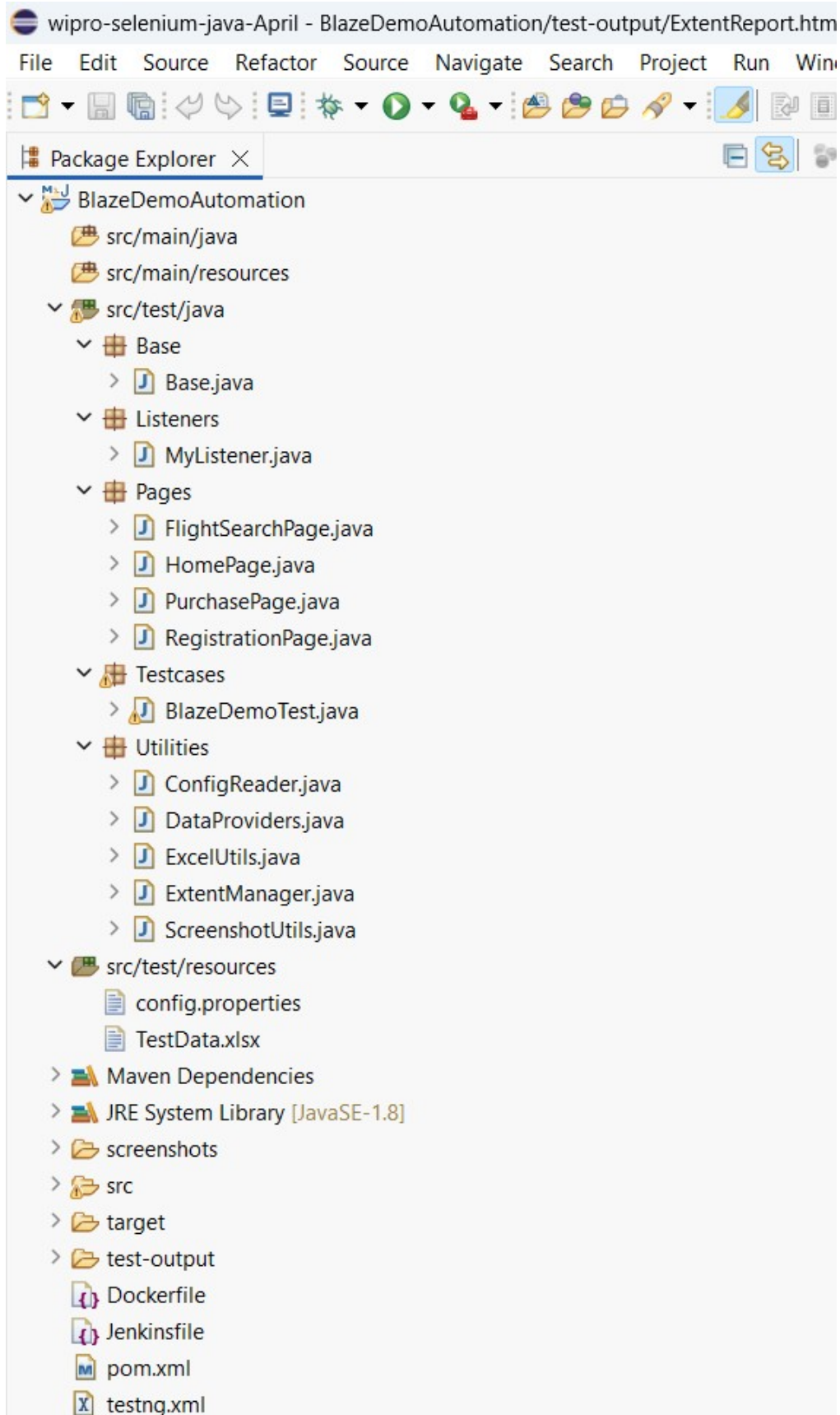
Source Code Documentation

Developed By: Ashish Shukla

Technology Stack: Java | Selenium WebDriver | TestNG | Maven |
Page Object Model (POM) | Jenkins | Docker | GitHub | Extent
Reports

Project Description: This document contains the complete source code structure, framework architecture, page object classes, utility classes, test scripts, listener implementation, Jenkins pipeline configuration, Docker integration, and reporting mechanism used in the BlazeDemo Flight Booking Automation Project.

1. Structure of Project



Base

Base.java Class

```
package Base;

import org.openqa.selenium.WebDriver;
import org.openqa.selenium.chrome.ChromeDriver;
import org.testng.annotations.AfterMethod;
import org.testng.annotations.BeforeMethod;
import org.openqa.selenium.chrome.ChromeOptions;
import Utilities.ConfigReader;
import io.github.bonigarcia.wdm.WebDriverManager;
import java.time.Duration;

public class Base {

    public static WebDriver driver;

    @BeforeMethod
    public void setup() {

        //driver = new ChromeDriver();
        WebDriverManager.chromedriver().setup();
        ChromeOptions options = new ChromeOptions();
        options.addArguments("--headless=new");
        options.addArguments("--no-sandbox");
        options.addArguments("--disable-dev-shm-usage");

        driver = new ChromeDriver(options);
        driver.manage().window().maximize();
    }
}
```

```
        driver.manage().timeouts()
        .implicitlyWait(Duration.ofSeconds(10));
```

```
        driver.get(ConfigReader.get("url"));
    }
}
```

```
@AfterMethod
```

```
public void tearDown() {
```

```
    if (driver != null) {
        driver.quit();
    }
}
}
```

MyListener.java class

```
package Listeners;
```

```
import org.testng.ITestContext;
```

```
import org.testng.ITestListener;
```

```
import org.testng.ITestResult;
```

```
import com.aventstack.extentreports.ExtentReports;
```

```
import com.aventstack.extentreports.ExtentTest;
```

```
import Base.Base;
```

```
import Utilities.ExtentManager;
```

```
import Utilities.ScreenshotUtils;
```

```
public class MyListener implements ITestListener {
```

```
    ExtentReports extent = ExtentManager.getInstance();
    ExtentTest test;
```

```
@Override
```

```
public void onStart(ITestContext context) {
    System.out.println("Test Execution Started");
}
```

```
@Override
```

```

    public void onFinish(ITestContext context) {

        extent.flush();

        System.out.println("Test Execution Finished");
    }

    @Override
    public void onTestStart(ITestResult result) {

        Object[] data = result.getParameters();

        if (data.length > 0) {

            test = extent.createTest(
                result.getName() + " - " + data[0].toString());

        } else {

            test = extent.createTest(result.getName());
        }

        System.out.println("Test Started: " + result.getName());
    }

    @Override
    public void onTestSuccess(ITestResult result) {

        test.pass("Test Passed");

        System.out.println("Test Passed: " + result.getName());
    }

    @Override
    public void onTestFailure(ITestResult result) {

        test.fail(result.getThrowable());

        System.out.println("Test Failed: " + result.getName());

        try {

            ScreenshotUtils.captureScreenshot(
                Base.driver,
                result.getName());

            System.out.println("Screenshot captured successfully");

        } catch (Exception e) {

            System.out.println("Failed to capture screenshot");
        }
    }

```

```

        e.printStackTrace();
    }
}

@Override
public void onTestSkipped(ITestResult result) {

    test.skip("Test Skipped");

    System.out.println("Test Skipped: " + result.getName());
}
}

```

Pages

FlightSearchPage

```

package Pages;

import org.openqa.selenium.By;
import org.openqa.selenium.JavascriptExecutor;
import org.openqa.selenium.WebDriver;
import org.openqa.selenium.WebElement;

import Utilities.ScreenshotUtils;

public class FlightSearchPage {

    WebDriver driver;

    public FlightSearchPage(WebDriver driver) {
        this.driver = driver;
    }

    By chooseFlight =
By.xpath("//input[@value='Choose This Flight']");

    public void highlightAndSelectFlight(String name) {

        WebElement btn =
driver.findElement(chooseFlight);

        JavascriptExecutor js =
(JavascriptExecutor) driver;

        js.executeScript(
"arguments[0].style.border='3px solid red';" , btn);

        try {
            Thread.sleep(1500);
        } catch (Exception e) {
            e.printStackTrace();
        }
    }
}

```

```

    }

    ScreenshotUtils.captureScreenshot(
        driver,
        "3_Flight_Button_Highlighted_" + name);

    btn.click();
    }
}

```

HomePage.java

```

package Pages;

import org.openqa.selenium.By;
import org.openqa.selenium.WebDriver;
import org.openqa.selenium.support.ui.Select;
import org.openqa.selenium.support.ui.WebDriverWait;
import org.openqa.selenium.support.ui.ExpectedConditions;

import java.time.Duration;

public class HomePage {

    WebDriver driver;
    WebDriverWait wait;

    public HomePage(WebDriver driver) {
        this.driver = driver;
        this.wait = new WebDriverWait(driver, Duration.ofSeconds(10));
    }

    // Locators
    By fromPort = By.name("fromPort");
    By toPort = By.name("toPort");
    By findFlightsBtn = By.xpath("//input[@value='Find Flights']");

    public void selectFromCity(String fromCity) {

        wait.until(
            ExpectedConditions.visibilityOfElementLocated(fromPort));

        Select from =
            new Select(driver.findElement(fromPort));

        from.selectByVisibleText(fromCity);
    }

    public void selectToCity(String toCity) {

        wait.until(

```

```

ExpectedConditions.visibilityOfElementLocated(toPort));

        Select to =
        new Select(driver.findElement(toPort));

        to.selectByVisibleText(toCity);
    }

    public void clickFindFlights() {

        wait.until(
ExpectedConditions.elementToBeClickable(findFlightsBtn));

        driver.findElement(findFlightsBtn).click();
    }

    // Optional - old method
    public void searchFlights(String fromCity, String toCity) {

        selectFromCity(fromCity);

        selectToCity(toCity);

        clickFindFlights();
    }
}

```

PurchasePage.java

```

package Pages;

import org.openqa.selenium.By;
import org.openqa.selenium.WebDriver;

public class PurchasePage {

    WebDriver driver;

    public PurchasePage(WebDriver driver) {

        this.driver = driver;

    }
}

```



```
By name = By.id("inputName");
```

```
By address = By.id("address");
```

```
By city = By.id("city");
```

```
By state = By.id("state");
```

```
By zip = By.id("zipCode");
```

```
By cardNumber = By.id("creditCardNumber");
```

```
By cardMonth = By.id("creditCardMonth");
```

```
By cardYear = By.id("creditCardYear");
```

```
By cardHolderName = By.id("nameOnCard");
```

```
By purchaseBtn = By.xpath("//input[@value='Purchase Flight']");
```

```
// FORM FILL ONLY
```

```
public void fillForm(
```

```
    String n,
```

```
    String a,
```

```
    String c,
```

```
    String s,
```

```
    String z,
```

```
    String cardNo,
```

```
    String month,
```

```
    String year,
```

```
    String holderName
```

```
) throws Exception {
```

```
    System.out.println("Name = " + n);
```

```
    System.out.println("Address = " + a);
```

```
    System.out.println("City = " + c);
```

```
System.out.println("State = " + s);  
System.out.println("Zip = " + z);
```

```
driver.findElement(name).clear();  
driver.findElement(name).sendKeys(n);  
Thread.sleep(500);
```

```
driver.findElement(address).clear();  
driver.findElement(address).sendKeys(a);  
Thread.sleep(500);
```

```
driver.findElement(city).clear();  
driver.findElement(city).sendKeys(c);  
Thread.sleep(500);
```

```
driver.findElement(state).clear();  
driver.findElement(state).sendKeys(s);  
Thread.sleep(500);
```

```
driver.findElement(zip).clear();  
driver.findElement(zip).sendKeys(z);  
Thread.sleep(500);
```

```
driver.findElement(cardNumber).clear();  
driver.findElement(cardNumber).sendKeys(cardNo);  
Thread.sleep(500);
```

```
driver.findElement(cardMonth).clear();  
driver.findElement(cardMonth).sendKeys(month);  
Thread.sleep(500);
```

```
        driver.findElement(cardYear).clear();
        driver.findElement(cardYear).sendKeys(year);
        Thread.sleep(500);
```

```
        driver.findElement(cardHolderName).clear();
        driver.findElement(cardHolderName).sendKeys(holderName);
```

```
        Thread.sleep(2000);
    }
}
```

```
// PURCHASE BUTTON CLICK
```

```
public void clickPurchase() {
```

```
    driver.findElement(purchaseBtn).click();
}
}
```

Registration Page.java

```
package Pages;
```

```
import org.openqa.selenium.By;
import org.openqa.selenium.WebDriver;
```

```
public class RegistrationPage {
```

```
    WebDriver driver;
```

```
    // Locators
```

```
    By txtName = By.name("name");
    By txtCompany = By.name("company");
```

```
By txtEmail = By.name("email");  
By txtPassword = By.name("password");  
By txtConfirmPassword = By.name("password_confirmation");  
By btnRegister = By.xpath("//button[@type='submit']");
```

```
// Constructor
```

```
public RegistrationPage(WebDriver driver) {  
    this.driver = driver;  
}
```

```
// Actions
```

```
public void enterName(String name) {  
    driver.findElement(txtName).sendKeys(name);  
}
```

```
public void enterCompany(String company) {  
    driver.findElement(txtCompany).sendKeys(company);  
}
```

```
public void enterEmail(String email) {  
    driver.findElement(txtEmail).sendKeys(email);  
}
```

```
public void enterPassword(String password) {  
    driver.findElement(txtPassword).sendKeys(password);  
}
```

```
public void enterConfirmPassword(String confirmPassword) {  
    driver.findElement(txtConfirmPassword).sendKeys(confirmPassword);  
}
```

```
public void clickRegisterButton() {  
    driver.findElement(btnRegister).click();  
}  
}
```

TestCases

BlazeDemoTest

```
package Testcases;  
  
import Base.Base;  
  
import org.openqa.selenium.By;  
import org.openqa.selenium.JavascriptExecutor;  
import org.testng.annotations.Test;  
  
import Pages.*;  
import Utilities.ExcelUtils;  
import Utilities.DataProviders;  
import Utilities.ScreenshotUtils;  
  
public class BlazeDemoTest extends Base {  
  
    String file = "src/test/resources/TestData.xlsx";  
  
    @Test(  
        dataProvider = "flightData",  
        dataProviderClass = DataProviders.class  
    )  
    public void fullFlowTest(  

```

```
        String name,  
        String country,  
        String city,  
        String state,  
        String zip,  
        String cardNo,  
        String month,  
        String year,  
        String cardName
```

```
    ) throws Exception {
```

```
        try {
```

```
System.out.println("===== TEST STARTED =====");
```

```
// STEP 1 : HOME PAGE
```

```
HomePage hp = new HomePage(driver);
```

```
System.out.println("STEP 1: Searching Flights");
```

```
hp.selectFromCity("Boston");
```

```
hp.selectToCity("London");
```

```
ScreenshotUtils.captureScreenshot(  
    driver,
```

```
    "1_Cities_Selected_" + name);
```

```
hp.clickFindFlights();
```

```
Thread.sleep(2000);
```

```
ScreenshotUtils.captureScreenshot(  
    driver,  
    "2_Flights_Found_" + name);
```

```
// STEP 2 : FLIGHT PAGE
```

```
System.out.println("STEP 2: Selecting Flight");
```

```
FlightSearchPage fp =  
    new FlightSearchPage(driver);
```

```
fp.highlightAndSelectFlight(name);
```

```
Thread.sleep(2000);
```

```
ScreenshotUtils.captureScreenshot(  
    driver,  
    "4_Purchase_Page_" + name);
```

```
// STEP 3 : PURCHASE PAGE
```

```
System.out.println("STEP 3: Filling Purchase Details");
```

```
PurchasePage pp =
```

```
new PurchasePage(driver);
```

```
pp.fillForm(  
    name,  
    country,  
    city,  
    state,  
    zip,  
    cardNo,  
    month,  
    year,  
    cardName  
);
```

```
Thread.sleep(3000);
```

```
ScreenshotUtils.captureScreenshot(  
    driver,  
    "5_Form_Filled_" + name);
```

```
pp.clickPurchase();
```

```
System.out.println("STEP 4: Final Confirmation Page");
```

```
Thread.sleep(3000);
```

```
ScreenshotUtils.captureScreenshot(  
    driver,  
    "6_Confirmation_" + name);
```



```
// STEP 5 : WRITE RESULT TO EXCEL
```

```
System.out.println("STEP 5: Writing Excel Data");
```

```
if (name.equals("Ashish")) {
```

```
    ExcelUtils.setCellData(  
        file,  
        "PurchaseData",  
        1,  
        9,  
        "SUCCESS");  
}
```

```
else if (name.equals("Virat")) {
```

```
    ExcelUtils.setCellData(  
        file,  
        "PurchaseData",  
        2,  
        9,  
        "SUCCESS");  
}
```

```
else if (name.equals("Rohit")) {
```

```
    ExcelUtils.setCellData(  
        file,  
        "PurchaseData",  
        3,
```

```

        9,
        "SUCCESS");
    }

    System.out.println(
"===== TEST COMPLETED SUCCESSFULLY =====");

    }

    catch (Exception e) {

        System.out.println(
"===== TEST FAILED =====");

        e.printStackTrace();

        ScreenshotUtils.captureScreenshot(
            driver,
            "FAILED_TEST_" + name);

        throw new RuntimeException(e);
    }
}
}
}

```

Utilities

ConfigReader.java

```
package Utilities;
```

```

import java.io.FileInputStream;

import java.util.Properties;


public class ConfigReader {


    public static Properties prop = new Properties();


    static {

        try {

            FileInputStream fis =
new FileInputStream("src/test/resources/config.properties");


            prop.load(fis);


        } catch (Exception e) {
            e.printStackTrace();
        }
    }


    public static String get(String key) {
        return prop.getProperty(key);
    }
}

```

DataProviders.java

```

package Utilities;

import org.testng.annotations.DataProvider;

public class DataProviders {

    @DataProvider(name = "flightData")
    public Object[][] getFlightData() throws Exception {

```

```

        String file =
"src/test/resources/TestData.xlsx";

        int rows =
ExcelUtils.getRowCount(
        file,
        "PurchaseData");

        Object[][] data =
new Object[rows][9];

        for (int i = 1; i <= rows; i++) {

            for (int j = 0; j < 9; j++) {

                data[i - 1][j] =
ExcelUtils.getCellData(
                file,
                "PurchaseData",
                i,
                j);
            }
        }

        return data;
    }
}

```

ExcelUtils.java

```

package Utilities;

import org.apache.poi.xssf.usermodel.XSSFSheet;
import org.apache.poi.xssf.usermodel.XSSFWorkbook;

import java.io.FileInputStream;
import java.io.FileOutputStream;

import org.apache.poi.ss.usermodel.Sheet;
import org.apache.poi.ss.usermodel.Workbook;

public class ExcelUtils {

    static XSSFWorkbook wb;

```

```

        static XSSFSheet sheet;

public static String getCellData(String path, String sheetName, int row, int col) throws
        Exception {

        FileInputStream fis = new FileInputStream(path);

        wb = new XSSFWorkbook(fis);

        sheet = wb.getSheet(sheetName);

        String value = sheet.getRow(row).getCell(col).toString();

        wb.close();

        return value;

    }

public static int getRowCount(String path,
        String sheetName) throws Exception {

        FileInputStream fis =
            new FileInputStream(path);

        Workbook wb =
            new XSSFWorkbook(fis);

        Sheet sheet =
            wb.getSheet(sheetName);

        System.out.println("Sheet Name = " + sheetName);

        if(sheet == null) {

        System.out.println("Sheet not found : " + sheetName);

        }
    }

```

```
        int rowCount =  
        sheet.getLastRowNum();
```

```
        wb.close();
```

```
        return rowCount;  
    }  
}
```

```
public static void setCellData(String path, String sheetName, int row, int col, String value)  
    throws Exception {
```

```
    FileInputStream fis = new FileInputStream(path);
```

```
    wb = new XSSFWorkbook(fis);
```

```
    sheet = wb.getSheet(sheetName);
```

```
    // □ FIX 1: If sheet is null
```

```
    if (sheet == null) {
```

```
        throw new RuntimeException("Sheet not found: " + sheetName);
```

```
    }
```

```
    // □ FIX 2: If row is null
```

```
    if (sheet.getRow(row) == null) {
```

```
        sheet.createRow(row);
```

```
    }
```

```
    // □ FIX 3: If cell is null
```

```
    if (sheet.getRow(row).getCell(col) == null) {
```

```
        sheet.getRow(row).createCell(col);
```

```
    }
```

```
sheet.getRow(row).getCell(col).setCellValue(value);
```

```
FileOutputStream fos = new FileOutputStream(path);
```

```
wb.write(fos);
```

```
fos.close();
```

```
wb.close();
```

```
}
```

```
}
```

ExtentManager.java

```
package Utilities;
```

```
import com.aventstack.extentreports.ExtentReports;
```

```
import com.aventstack.extentreports.reporter.ExtentSparkReporter;
```

```
public class ExtentManager {
```

```
private static ExtentReports extent;
```

```
public static ExtentReports getInstance() {
```

```
if (extent == null) {
```

```
    ExtentSparkReporter spark =
```

```
new ExtentSparkReporter("test-output/ExtentReport.html");
```

```
spark.config().setReportName("BlazeDemo Automation Report");
```

```
spark.config().setDocumentTitle("Automation Execution Report");
```

```
extent = new ExtentReports();
```

```
extent.attachReporter(spark);
```

```
extent.setSystemInfo("Tester", "Ashish Shukla");
```

```
extent.setSystemInfo("Project", "BlazeDemo Automation");
```

```
}
```

```
return extent;
```

```
}
```

```
}
```

ScreenShotUtil.java

```

package Utilities;

import org.apache.commons.io.FileUtils;
import org.openqa.selenium.*;

import java.io.File;

public class ScreenshotUtils {

    public static void captureScreenshot(WebDriver driver, String name) {

        try {
            File src = ((TakesScreenshot) driver).getScreenshotAs(OutputType.FILE);

            File dest = new File(
                System.getProperty("user.dir") +
                "/screenshots/" + name + ".png"
            );

            FileUtils.copyFile(src, dest);

        } catch (Exception e) {
            e.printStackTrace();
        }
    }
}

```

DockerFile

FROM maven:3.9.6-eclipse-temurin-21

RUN apt-get update && \
apt-get install -y wget gnupg unzip && \


```
wget -q -O - https://dl.google.com/linux/linux_signing_key.pub | gpg --dearmor -o  
/usr/share/keyrings/google.gpg && \  
echo "deb [arch=amd64 signed-by=/usr/share/keyrings/google.gpg]  
http://dl.google.com/linux/chrome/deb/ stable main" > /etc/apt/sources.list.d/google.list && \  
apt-get update && \  
apt-get install -y google-chrome-stable
```

WORKDIR /app

COPY . .

RUN mvn clean install -DskipTests

CMD ["mvn","test"]

JenkinsFile

```
pipeline {  
    agent any  
  
    tools {  
        jdk 'JDK17'  
        maven 'Maven'  
    }  
  
    stages {  
        stage('Checkout') {  
            steps {  
                checkout scm  
            }  
        }  
  
        stage('Clean Project') {  
            steps {  
                bat 'mvn clean'  
            }  
        }  
  
        stage('Build Project') {  
            steps {  
                bat 'mvn compile'  
            }  
        }  
  
        stage('Run TestNG Tests') {  
            steps {  
                bat 'mvn test'  
            }  
        }  
    }  
}
```

```

    }
  }
  stage('Docker Build') {
    steps {
      bat 'docker build -t blazedemo-automation .'
    }
  }

  stage('Docker Run') {
    steps {
      bat 'docker run --rm blazedemo-automation'
    }
  }

  post {

    always {

      archiveArtifacts artifacts: 'screenshots/*.png', allowEmptyArchive: true

      archiveArtifacts artifacts: 'test-output/**/*.*', allowEmptyArchive: true
    }

    success {
      echo 'BlazeDemo Automation Executed Successfully'
    }

    failure {
      echo 'BlazeDemo Automation Failed'
    }
  }
}

```

Pom.xml

```

<project xmlns="http://maven.apache.org/POM/4.0.0"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 https://maven.apache.org/xsd/maven-
    4.0.0.xsd">
  <modelVersion>4.0.0</modelVersion>
  <groupId>Blazedemo</groupId>
  <artifactId>Blazedemo</artifactId>
  <version>0.0.1-SNAPSHOT</version>
  <dependencies>
    <!-- Source: https://mvnrepository.com/artifact/org.testng/testng -->
    <dependency>
      <groupId>org.testng</groupId>
      <artifactId>testng</artifactId>
      <version>7.12.0</version>
      <scope>test</scope>
    </dependency>
  </dependencies>
</project>

```

```

        </dependency>
<!-- Source: https://mvnrepository.com/artifact/org.seleniumhq.selenium/selenium-java -->
        <dependency>
        <groupId>org.seleniumhq.selenium</groupId>
        <artifactId>selenium-java</artifactId>
        <version>4.44.0</version>
        <scope>compile</scope>
        </dependency>
        <dependency>
        <groupId>org.apache.poi</groupId>
        <artifactId>poi-ooxml</artifactId>
        <version>5.2.5</version>
        </dependency>
<!-- WebDriverManager -->
        <dependency>
        <groupId>io.github.bonigarcia</groupId>
        <artifactId>webdrivermanager</artifactId>
        <version>6.1.0</version>
        </dependency>
        <dependency>
        <groupId>commons-io</groupId>
        <artifactId>commons-io</artifactId>
        <version>2.15.1</version>
        </dependency>
<!-- LOG4J CORE (FIXES YOUR ERROR) -->
        <dependency>
        <groupId>org.apache.logging.log4j</groupId>
        <artifactId>log4j-core</artifactId>
        <version>2.23.1</version>
        </dependency>

        <dependency>
        <groupId>org.apache.logging.log4j</groupId>
        <artifactId>log4j-api</artifactId>
        <version>2.23.1</version>
        </dependency>

<!-- OPTIONAL BUT SAFE -->
        <dependency>
        <groupId>org.apache.logging.log4j</groupId>
        <artifactId>log4j-slf4j2-impl</artifactId>
        <version>2.23.1</version>
        </dependency>
        <dependency>
        <groupId>com.aventstack</groupId>
        <artifactId>extentreports</artifactId>
        <version>5.1.1</version>
        </dependency>
    </dependencies>
    <build>
    <plugins>

```

```

        <plugin>
<groupId>org.apache.maven.plugins</groupId>
<artifactId>maven-surefire-plugin</artifactId>
    <version>3.5.3</version>
    <configuration>
    <suiteXmlFiles>
<suiteXmlFile>testng.xml</suiteXmlFile>
    </suiteXmlFiles>
    </configuration>
    </plugin>
</plugins>
</build>
</project>

```

Testng.xml

```

<!DOCTYPE suite SYSTEM "https://testng.org/testng-1.0.dtd">

<suite name="BlazeDemo Suite" parallel="false">

    <listeners>
<listener class-name="Listeners.MyListener"/>
    </listeners>

    <test name="End To End Test">

        <classes>
<class name="Testcases.BlazeDemoTest"/>
        </classes>

    </test>

</suite>

```